Graduate Algorithms

Applications of Flows

02 November 2026

Participation in a game, any kind of game, gives you new ways of perceiving others. Victory only gives you new ways of perceiving yourself.

— *Sally Roony*

Remark

Recall, we can solve a max-flow problem:

- in $O(nm^2)$ time using Edmond-Karp (irrespective of capacities or flow)
- in $O(m\ |f|)$ time using flow template aka Ford-Fulkerson Method.
- in $O((m+n)m\log(|f|))$ using Ford-Fulkerson algorithm

In some problems, $|f| \leq Cn$ for some constant C, in that case we can solve in $O(m\ |f|) = O(m \cdot (Cn)) = O(mn)$ using Ford-Fulkerson Method (which in this case is faster than Edmond-Karp).

§1 Classics

Bipartite Matching

Given n_1 students and n_2 different food items in the mess, we can form a bipartite graph $G = (V_1 \cup V_2, E)$ where $(u, v) \in E$ implies that the student u likes the food item v.

Assign food items to students, maximizing the number of students who receive a food item they like (you don't need to assign all food items, a student might not be assigned a food item which they don't like).

Formally, we want a the maximal matching M of G. A matching of G is a subset of edges in G such that no vertex is incident to more than 1 edge included in the matching.

Hint: The idea is that a vertex or an edge can only participate in atmost one matching. Can we enforce this using flows?

Hint: Consider making the graph directed. Now if I add a (dummy) source vertex and join it to all the students and a (dummy) target vertex and join it to all the food; and the capacity of all edges is 1, what would the max flow be?

Assignment Problem

Given a set X of students and a set Y of ice cream tubs.

A student $x \in X$ can only eat $c(x)$ scoops of ice cream. An ice cream tub $y \in Y$ can only has $c(y)$ scoops of ice cream.

Finally, each pair of student and ice-cream (x, y) can only be matched $c(x, y)$ times (student x only wants $c(x, y)$ scoops of flavour y).

Use up as many scoops as possible!

Hint: Try modifying the above construction. How do we enforce $c(x)$ and $c(y)$ although?

Decomposition of Flows

Given a flow network $G = (V, E)$, $c : E \to \mathbb{R}_{\geq 0}$, $s, t \in V$. Let f, f' be flows on G and g be flow on G_f (residual graph).

Prove that the flow $f' - f$ on G_f has $\text{cost}(f' - f) = \text{cost}(f') - \text{cost}(f)$.

(ii) Prove that the flow $f + g$ on G has $\text{cost}(f + g) = \text{cost}(f) + \text{cost}(g)$.

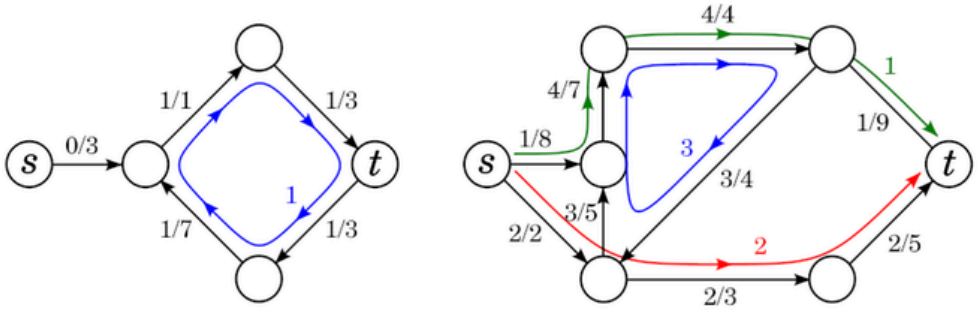
(iii) Prove that $\left(G_f\right)_g = G_{f+g}$ if we count parallel edges with same cost as equal.

(iv) Using the above, prove the **flow decomposition theorem**: Given a flow network (G, c, s, t); the flow can be decomposed into paths and cycles.

That is there exists a finite set $\mathcal{P}$ of paths from s to t and a finite set of cycles $\mathcal{C}$ such that each path and cycle have a positive weight w and

$$f(e) = \sum_{P \in \mathcal{P}, e \in P} w(P) + \sum_{C \in \mathcal{C}, e \in C} w(C)$$

.



Minimum Cost Maximum Flow

Given a flow network $G = (V, E)$, $c : E \to \mathbb{R}_{\geq 0}$, $s, t \in V$ and a cost function $\alpha : E \to \mathbb{R}$ the cost of a flow f in g is:

$$\text{cost}(f) = \sum_{e \in E} f(e)a(e)$$

The minimum cost maximum flow problem is to find a flow f in G such that the value of the flow is maximized, and among all possible solutions with maximum value, to find the one with minimum cost.

Assume that G doesn't have negative cycles wrt α. We want to show that this algorithm works:

Min Cost Max Flow

```
Initialize f by setting f(e) ← 0 for all edges
While t is reachable from s in G_f (residual graph):
    Let P be the the shortest path from s to t (wrt α)
    Let γ be the minimum capacity among the edges of P in G_f.
    Define Δf by putting γ > 0 on every edged in P.
    Let f ← f + Δf
return f
```

(i) Show that an iteration of this algorithm doesn't end up creating negative cycles in G_f. You might want to use the flow decomposition theorem.

(ii) Show that the returned f will be the minimum cost flow. Note, we can be sure the flow is maximum as we are following the max-flow template.

Min Cost Fixed Flow

Same setup as Min Cost Max Flow, but you are given the amount of flow a you need to achieve and you need to minimize the cost among flows achieving it. That is find $\min_{|f| \geq a} \text{cost}(f)$.

Hint: If a push would take us above a, just push as much to reach a and then terminate. Why is this min cost?

Intersection of Min Cut

Given a flow network where all minimum cuts have a nonempty intersection, prove that there exists an edge that is in every minimum cut. The intersections are taken edge-wise.

Hint: Consider G_f for max flow f. Can you see two min cuts (which are unique upto relabelling)?

Hint: Think about their intersection now. You'll either get a contradiction or (neighboring) vertices u and v such that u is always reachable from s and v is never reachable from s in G_f.

§2 Multiple Routing

Two Routes with no overlapping edges

Gojo wants Yuji and Megumi to travel from Sendai to Tokyo. However, if Yuji or Megumi end up on the same road, they will inevitably start racing and crash their bikes.

So, Gojo, given the road network is a directed graph $G = (V, E)$ wants to find two routes from s (Sendai) to t (Tokyo) such that none of the edges is used in both the routes.

Give an algorithms to do so in $O(nm)$ time or report that it is not possible.

Two Routes with no overlapping vertices

Gojo realized that Yuji and Megumi will end up doing something stupid if they ever end up in the same city (vertex).

So, Gojo, given the road network is a directed graph $G = (V, E)$ wants to find two routes from s (Sendai) to t (Tokyo) such that none of the vertices is used in both the routes.

Give an algorithms to do so in $O(n(n + m))$ time or report that it is not possible.

Multiple Routes with no overlapping vertices

Gojo realized that this is true for all the k sorcerers.

So, Gojo, given the road network is a directed graph $G = (V, E)$ wants to find k routes from s (Sendai) to t (Tokyo) such that none of the vertices is used in more than 1 of these routes.

Give an algorithms to do so in $O((n + m)nk)$ time or report that it is not possible.

Shortest multiple routes with no overlapping vertices

Gojo also doesn't want these people to take ages in getting back. Let $\alpha : E \to \mathbb{R}_{\geq 0}$ be the time to traverse an edge (length of road).

So now he wants to find k routes from s to t such that no vertex is used in more than one route and the total length of routes is minimized.

Hint: Think about min cost fixed flow!

§3 Tuple Selection Problems

The bipartite maximum matching and assignment problems are examples of a larger class of problems which Jeff Ericson calls "Tuple Selection".

Tuple Selection Problem

Given finite sets $X_1, X_2, ..., X_d$; we want to select the largest possible set of d-tuples of the form $(x_1, x_2, ..., x_d)$ where $x_i \in X_i$ subject to:

- For each i, $x \in X_i$ can only appear in $c(x)$ many tuples
- For each i, $x \in X_i$ and $y \in X_{i+1}$ can appear together in atmost $c(x, y)$ many tuples.

Where $c(x)$ and $c(x, y)$ are positive integers or ∞.

We can see the maximum bipartite matching problem was a $d = 2$, $c(x) = 1$ and $c(x, y) = 1$ if xy is an edge in the original graph and 0 otherwise.

Similarly, assignment problem is the general $d = 2$ case.

Tuple Selection using Flows

Show that the tuple selection problem can be solved in $O(nm^2)$ time where

$$n = \sum_i |X_i| \quad m = \prod_i |X_i|$$

Exam Scheduling

You have to schedule n exams in r rooms on d dates to be invigilated by p invigilators upto to the constraints:

- Each exam has to be scheduled only once.
- No two exams can happen in the same room, on the same date
- No invigilator can invigilate more than one exam on the same day.
- Every invigilator has a list of days on which they are available. They can't invigilate on any other days.
- No invigilator can invigilate more than 3 exams.

Reduce this problem to Tuple Selection.

Tournament Elimination

n teams are playing in a tournament. Team i has won $W[i]$ games and has $M[i, j]$ matches remaining against team j.

Design an algorithm, that given a team i, tells if it is possible for team i to win the tournament (that is have or be tied for the highest number of wins after all remaining matches are done). We are expecting a $O(n^6)$ algorithm.

Hint: Why is this question in this problem sheet? And why in this section?

Hogwarts Renovation Committee

The Hogwarts School of Witchcraft and Wizardry has decided to form a committee to renovate the castle. Exactly one professor must be selected from each of the school's academic departments.

Some professors teach in multiple departments. However, each professor can represent only one department on the committee.

For example, if Professor Snape teaches both Potions and Defence Against the Dark Arts, and he is selected as the representative for Potions, then a different professor must represent Defence Against the Dark Arts.

Additionally, the committee must contain an equal number of professors from each of the houses: Hufflepuff, Ravenclaw, Gryfindor and Slytherin.

Fortunately, the total number of departments is a multiple of 4, say $4k$.

Design an efficient algorithm to select the committee members or tells that it is impossible.

Election

Consider an election between two candidates, A and B, with n voters.

Each voter i has a preferred candidate $C_i \in \{A, B\}$ and a preference penalty $p_i \in \mathbb{Z}$, representing the cost of voting against their preference.

Voter relationships are represented by an undirected weighted graph $G = (V, E)$, where each vertex represents a voter and each edge (i, j) has weight $w_{ij} \in \mathbb{Z}$, representing the strength of their friendship.

Each voter incurs:

- A penalty of p_i for voting against their preferred candidate.
- A penalty of w_{ij} if they vote differently from a friend j.

Design an efficient algorithm that, given G, the voter preferences C_i, and the penalties p_i and w_{ij}, determines the voting assignment that minimizes the total penalty and identifies the winning candidate.

§4 Graph Algorithms

Max Weight Closure

Given a directed graph $G = (V, E)$, a subset $S \subseteq V$ is called a **closure** if no edge enters S from outside it. Formally, $\forall v \in S, N_{\text{in}}(v) \cap (V \setminus S) = \varnothing$.

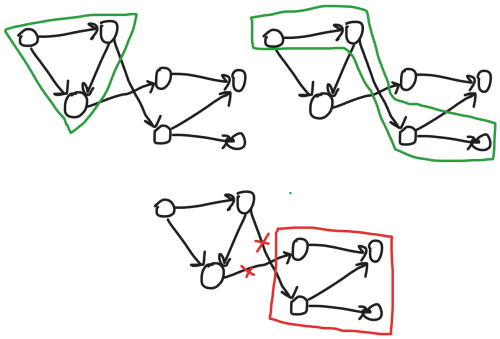


Figure 1: The top two green sets are valid closures; the bottom one is not (note the crossed incoming edges).

Monster's University allows students to freely choose courses for their degree, subject to two rules:

- To take a course, you must also take all its prerequisites.
- The selected courses must form a closure.

The prerequisite structure is represented by a directed acyclic graph $G = (C, E)$, where $(u, v) \in E$ means that u is a prerequisite for v.

Each course $v \in C$ has a market value $m(v) \in \mathbb{R}$. A course may even have negative value, perhaps because it makes you more likely to pursue a PhD, making companies wary of hiring you!

Design an algorithm that given G and m, finds the valid degree (a closure) that maximises the total market value:

$$\max_{S \subseteq C} \sum_{v \in S} m(v)$$

Disjoint Path Cover in DAGs

A path cover for a directed-graph G is a set of (directed) paths such that every vertex of G lies on atleast one of the paths.

We call a path cover disjoint if every vertex lies on exactly one path.

Given a directed acyclic graph, reduce finding the disjoint path cover to a bipartite matching problem (and hence, a flow problem!) and solve in $O(nm)$ time.

Hint: What could the matching represent? What about a being matched to b represents there is a path where $a \to b$ (b comes after a)?

Hint: We claim we can cover in k disjoint paths if there is a matching of size $|V| - k$ in the flow graph we make...

Sam I Am

Sam is facing a temple which can be described by a $m \times n$ grid and he has the locations of all enemies in the temple (each location can be thought of as the intersection of a row and a column in this grid).

All of a sudden, he realizes that he can kill the enemies without entering the temple using the great cannon ball which spits out a gigantic ball bigger than him killing anything it runs into and keeps on rolling until it finally explodes.

But the cannonball can only shoot horizontally or vertically and all the enemies along the path of that cannon ball will be killed.

Sam wants to know the minimum number of cannon balls and the positions from which he can shoot the cannonballs to eliminate all enemies from outside that temple.

Hint: Consider the graph $G = (V, E)$ where $V = \{R_1, R_2, ..., R_m, C_1, C_2, ..., C_n\}$ and $(R_i, C_j) \in E \iff$ there is an enemy at the intersection of row i and column j. What would a valid (not minimal, just kills all enemies) shooting scheme be equivalent to? What does a matching in G corresponds to in the problem instance?

Hint: In the residual graph, consider the row vertices that are unreachable from s and column vertices from where it is impossible to reach t. Let's name these misfits. Using the maxFlow-minCut duality, what can be said about misfits?

Largest Common Rooted Tree

A rooted tree is a directed acyclic graph, in which every vertex has exactly one incoming edge, except for the root, which has no incoming edges Equivalently, a rooted tree consists of a root vertex, which has edges pointing to the roots of zero or more smaller rooted trees.

Describe a polynomial-time algorithm to compute, given two rooted trees A and B, the largest common rooted subtree of A and B.

Hint: An approach would be to compute $\text{dp}[u, v]$ which would be the largest common rooted subtree between $u \in A$ (and it's descendants) and $v \in B$ (and it's descendants) where for all vertices u and v using dynamic programming...

Hint: Sadly, this is not tht simple. If the number of children was bounded or there was some ordering over them, it might have worked. Given arbitrary number of children, we need a mapping from children of u to those of B (or shall I say, matching...)

§5 Fun Problem!

Rounding a Matrix

Suppose we are given an array $A[1...m][1...n]$ of non-negative real numbers. We want to round A to an integer matrix, by replacing each entry x in A with either $\lfloor x \rfloor$ or $\lceil x \rceil$, without changing the sum of entries in any row or column of A. For example:

$$\begin{pmatrix} 1.2 & 3.4 & 2.4 \\ 3.9 & 4.0 & 2.1 \\ 7.9 & 1.6 & 0.5 \end{pmatrix} \mapsto \begin{pmatrix} 1 & 4 & 2 \\ 4 & 4 & 2 \\ 8 & 1 & 1 \end{pmatrix}$$

(a) Describe and analyze an efficient algorithm that either rounds A in this fashion, or reports correctly that no such rounding is possible.

(b) Prove that a legal rounding is possible if and only if the sum of entries in each row is an integer, and the sum of entries in each column is an integer. In other words, prove that either your algorithm from part (a) returns a legal rounding, or a legal rounding is obviously impossible.

Extra: Suppose we are guaranteed that none of the entries in the input matrix A is an integer Describe and analyze an even faster algorithm that either rounds A or reports correctly that no such rounding is possible. Your algorithm must run in $O(mn)$ time.

Abusing Trader Problem

You (unilaterally) control the price of an asset M. A price movement $\mathcal{M} = (m_1, m_2, ...)$ is a sequence of ± 1 such that the price/value of M on day j is $\sum_{i=1}^{j} m_i$.

A trade is a tuple (w, l, f) where the trader makes $f(w)$ profit if M reaches w before reaching l and makes a loss of $f(l)$ if it reaches l before reaching w. It has value 0 otherwise.

Given a set of trades $\mathcal{T}$, choose a movement $\mathcal{M}$ to maximize trader loss.[1]

[1]There is a $O(n^2)$ algorithm where n is the number of trades. That is much harder to find but very beautiful. Check out the paper *Lafond's "How Brokers Can Optimally Abuse Traders", FUN 2022.*